PRAKTIKUM SISTEM OPERASI

MODUL 7

Semaphore

Oleh:

Candra Dinata - 2311104061

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

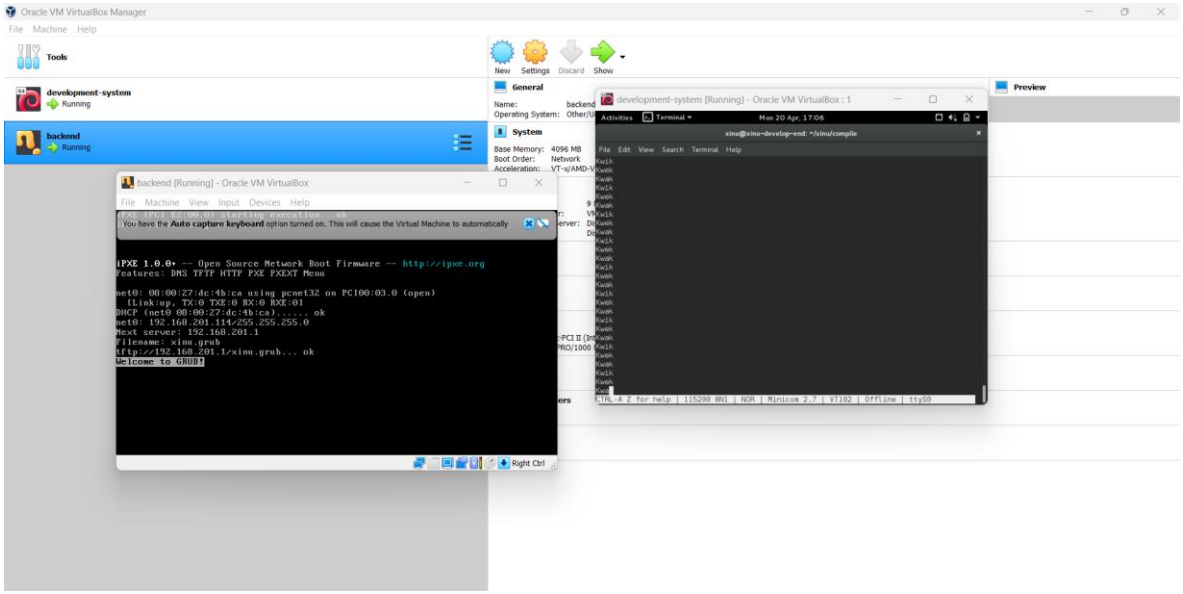
UNIVERSITAS TELKOM

2026

JURNAL

1. **Kode**

```
#include <xinu.h>

sid32 s1, s2, s3;

process p1(void) {
   while (1) {
      wait(s1);
      printf("Kwak\n");
      signal(s2);
   }
}

process p2(void) {
   while (1) {
      wait(s2);
      printf("Kwik\n");
      signal(s3);
   }
}

process p3(void) {
   while (1) {
      wait(s3);
      printf("Kwek\n");
      signal(s1);
   }
}

process main(void) {

   s1 = semcreate(1);
   s2 = semcreate(0);
   s3 = semcreate(0);

   resume(create(p1, 1024, 20, "P1", 0));
   resume(create(p2, 1024, 20, "P2", 0));
   resume(create(p3, 1024, 20, "P3", 0));

   return OK;
}
```

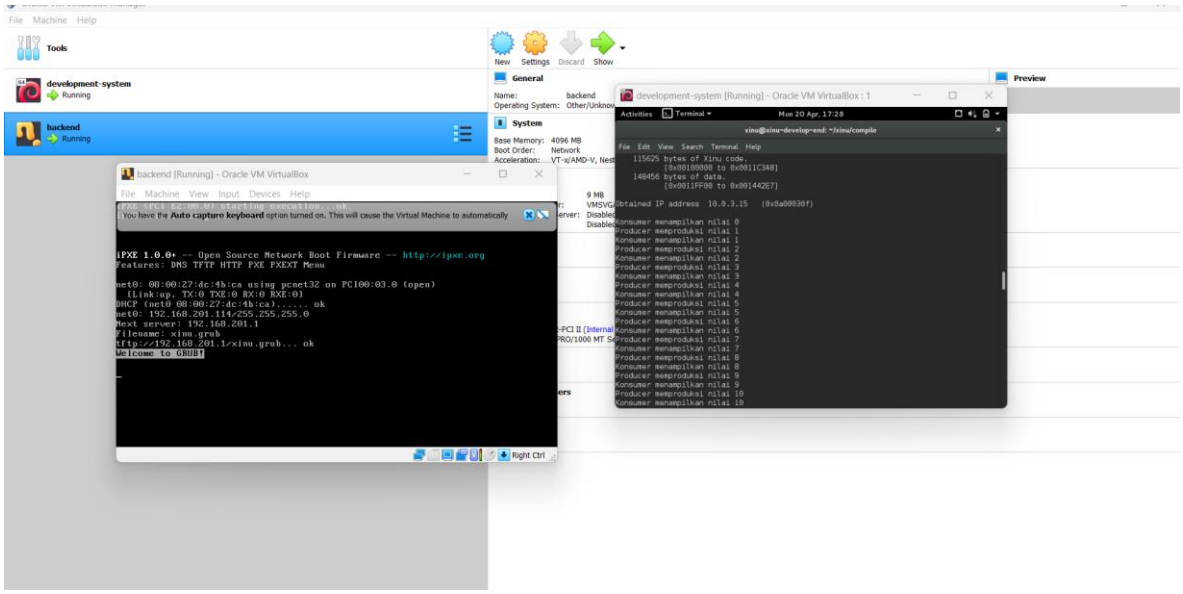


Program di atas merupakan implementasi sinkronisasi proses menggunakan semaphore pada sistem operasi Xinu. Terdapat tiga proses, yaitu p1, p2, dan p3, yang masing-masing bertugas mencetak teks “Kwak”, “Kwik”, dan “Kwek” secara berurutan dan berulang. Untuk mengatur urutan eksekusi tersebut, digunakan tiga semaphore bertipe sid32, yaitu s1, s2, dan s3. Pada fungsi main, ketiga semaphore diinisialisasi dengan semcreate, di mana s1 bernilai awal 1 (sehingga proses p1 dapat berjalan pertama), sedangkan s2 dan s3 bernilai 0 (menunggu giliran). Ketiga proses kemudian dibuat menggunakan create() dan dijalankan dengan resume() sehingga berjalan secara konkuren.

Mekanisme sinkronisasi terjadi melalui operasi wait() dan signal() pada masing-masing proses. Proses p1 akan menunggu semaphore s1, lalu mencetak “Kwak” dan memberikan sinyal ke s2 agar proses p2 dapat berjalan. Selanjutnya, p2 menunggu s2, mencetak “Kwik”, lalu memberi sinyal ke s3 untuk mengaktifkan p3. Proses p3 kemudian menunggu s3, mencetak “Kwek”, dan mengembalikan sinyal ke s1 sehingga siklus kembali ke p1. Dengan pola ini, ketiga proses tidak berjalan secara acak, melainkan bergantian secara teratur membentuk urutan “Kwak → Kwik → Kwek”.

2. **Kode**

```
#include <xinu.h>

sid32 empty, full;
int buffer;

process producer(void) {
   int i;
   for (i = 1; i <= 1000; i++) {
      wait(empty);

      buffer = i;
      printf("Producer memproduksi nilai %d\n", i);

      signal(full);
   }
   return OK;
}

process consumer(void) {
   int i;
   for (i = 1; i <= 1000; i++) {
      wait(full);

      printf("Konsumer menampilkan nilai %d\n", buffer);

      signal(empty);
   }
   return OK;
}

/* ===== MAIN ===== */
process main(void) {

   empty = semcreate(1);
   full  = semcreate(0);

   resume(create(producer, 1024, 20, "Producer", 0));
   resume(create(consumer, 1024, 20, "Consumer", 0));

   while (1) {
      sleep(1);
   }

   return OK;
}
```



Program di atas merupakan implementasi masalah klasik producer-consumer menggunakan mekanisme sinkronisasi semaphore pada sistem operasi Xinu. Terdapat dua proses utama, yaitu producer yang bertugas menghasilkan data berupa bilangan dari 1 hingga 1000, dan consumer yang bertugas mengambil serta menampilkan data tersebut. Data sementara disimpan pada sebuah variabel bersama (buffer) yang berperan sebagai media komunikasi antar proses. Untuk mengatur akses terhadap buffer agar tidak terjadi konflik, digunakan dua semaphore bertipe sid32, yaitu empty dan full. Pada fungsi main, semaphore empty diinisialisasi dengan nilai 1 (menandakan buffer kosong dan siap diisi), sedangkan full diinisialisasi dengan nilai 0 (menandakan belum ada data yang bisa dikonsumsi). Kedua proses kemudian dibuat menggunakan create() dan dijalankan secara konkuren dengan resume().

Mekanisme sinkronisasi terjadi melalui operasi wait() dan signal() pada kedua proses. Pada proses producer, sebelum menulis ke buffer, dilakukan wait(empty) untuk memastikan buffer dalam keadaan kosong. Setelah itu, nilai dimasukkan ke buffer dan ditampilkan, kemudian signal(full) dipanggil untuk memberi tanda bahwa data siap dikonsumsi. Sebaliknya, proses consumer akan melakukan wait(full) untuk menunggu hingga data tersedia di buffer, kemudian membaca dan menampilkan nilai tersebut, lalu memanggil signal(empty) untuk menandakan bahwa buffer kembali kosong dan siap diisi oleh producer. Dengan mekanisme ini, kedua proses berjalan secara bergantian dan terkoordinasi tanpa saling mengganggu.